

JavaScript Array Method - concat()

The `concat()` method is one of the most useful array methods in JavaScript. It is used to **combine two or more arrays** into a **new array**.

Unlike `push()` or `splice()`, the `concat()` method **does not modify the original array**.

Table of Contents

- What is concat()?
- Why Use concat()?
- Syntax
- Parameters
- Return Value
- How concat() Works
- Basic Examples
- Multiple Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is concat()?

Definition

The `concat()` method combines two or more arrays and returns a **new array**.

The original arrays remain unchanged.

Why Use concat()?

We use `concat()` when we need to:

- Merge arrays

- Combine API data
- Join product lists
- Merge student lists
- Keep the original arrays unchanged

Syntax

```
array1.concat(array2);  
  
array1.concat(array2, array3);  
  
array1.concat(value1, value2);
```

Parameters

Parameter	Description
array2	Another array to combine
array3	Additional array (Optional)
value	Individual value(s) (Optional)

Return Value

Returns a **new array** containing all merged values.

The original arrays are **not modified**.

How concat() Works

Array 1

```
["Apple", "Mango"]
```

-

Array 2

```
["Orange", "Banana"]
```

↓

```
concat()
```

↓

New Array

```
["Apple", "Mango", "Orange", "Banana"]
```

Example 1 (Basic)

```
const fruits1 = [
  "Apple",
  "Mango"
];

const fruits2 = [
  "Orange",
  "Banana"
];

const allFruits = fruits1.concat(fruits2);

console.log(allFruits);
```

Output

```
["Apple", "Mango", "Orange", "Banana"]
```

Explanation

- `fruits1` and `fruits2` are merged.
- A new array is created.
- The original arrays remain unchanged.

Example 2 (Three Arrays)

```
const a = [1,2];  
  
const b = [3,4];  
  
const c = [5,6];  
  
const result = a.concat(b,c);  
  
console.log(result);
```

Output

```
[1,2,3,4,5,6]
```

Example 3 (Array + Values)

```
const numbers = [  
  
  10,
```

```
    20  
  
];  
  
const result = numbers.concat(  
  
    30,  
  
    40  
  
);  
  
console.log(result);
```

Output

```
[10,20,30,40]
```

Example 4 (Empty Array)

```
const numbers = [];  
  
const result = numbers.concat(  
  
    100,  
  
    200  
  
);  
  
console.log(result);
```

Output

```
[100,200]
```

Example 5 (Strings)

```
const first = [
  "HTML",
  "CSS"
];

const second = [
  "JavaScript",
  "React"
];

const course = first.concat(second);

console.log(course);
```

Output

```
[
  "HTML",
  "CSS",
  "JavaScript",
  "React"
]
```

Example 6 (Objects)

```
const users1 = [
  {
    name: "John"
```

```
    }  
  ];  
  
  const users2 = [  
    {  
      name: "Sara"  
    }  
  ];  
  
  const users = users1.concat(users2);  
  
  console.log(users);
```

Output

```
[  
  { name: "John" },  
  { name: "Sara" }  
]
```

Example 7 (Original Arrays Stay Unchanged)

```
const a = [1,2];  
  
const b = [3,4];  
  
const result = a.concat(b);  
  
console.log(a);  
  
console.log(b);  
  
console.log(result);
```

Output

```
[1,2]
[3,4]
[1,2,3,4]
```

Example 8 (Nested Arrays)

```
const arr1 = [
  [1,2]
];

const arr2 = [
  [3,4]
];

const result = arr1.concat(arr2);

console.log(result);
```

Output

```
[
  [1,2],
  [3,4]
]
```

Real Project Example 1

Shopping Cart

```
const cart1 = [  
  "Laptop",  
  "Mouse"  
];  
  
const cart2 = [  
  "Keyboard",  
  "Monitor"  
];  
  
const cart = cart1.concat(cart2);  
  
console.log(cart);
```

Output

```
[  
  "Laptop",  
  "Mouse",  
  "Keyboard",  
  "Monitor"  
]
```

Real Project Example 2

Student List

```
const sectionA = [
```

```
    "Rahim",  
  
    "Karim"  
];  
  
const sectionB = [  
  
    "Rokon",  
  
    "Hasan"  
];  
  
const students = sectionA.concat(sectionB);  
  
console.log(students);
```

Output

```
[  
  "Rahim",  
  "Karim",  
  "Rokon",  
  "Hasan"  
]
```

Real Project Example 3

API Data Merge

```
const page1 = [  
  
  1,  
  
  2,  
  
  3  
];
```

```
const page2 = [
  4,
  5,
  6
];

const data = page1.concat(page2);

console.log(data);
```

Output

```
[1,2,3,4,5,6]
```

Real Project Example 4

Product Categories

```
const electronics = [
  "Laptop",
  "Phone"
];

const accessories = [
  "Mouse",
  "Keyboard"
];

const products = electronics.concat(accessories);
```

```
console.log(products);
```

Output

```
[  
  "Laptop",  
  "Phone",  
  "Mouse",  
  "Keyboard"  
]
```

Real Project Example 5

Programming Languages

```
const frontend = [  
  "HTML",  
  "CSS",  
  "JavaScript"  
];  
  
const backend = [  
  "Node.js",  
  "Express"  
];  
  
const fullStack = frontend.concat(backend);  
  
console.log(fullStack);
```

Output

```
[  
  "HTML",  
  "CSS",  
  "JavaScript",  
  "Node.js",  
  "Express"  
]
```

Best Practices

Use `concat()` when you want to keep the original arrays unchanged.

Store the returned array.

```
const result = arr1.concat(arr2);
```

Use meaningful variable names.

```
mergedData  
  
allProducts  
  
students  
  
users
```

Use `concat()` instead of manually looping when merging arrays.

Common Mistakes

Mistake 1

Thinking `concat()` changes the original array.

Wrong

```
const a = [1,2];  
  
a.concat([3,4]);  
  
console.log(a);
```

Output

```
[1,2]
```

Mistake 2

Not storing the returned array.

Wrong

```
arr1.concat(arr2);
```

Correct

```
const merged = arr1.concat(arr2);
```

Mistake 3

Confusing `concat()` with `push()` .

```
push()
```

Adds elements to the same array.

```
concat()
```

Creates a **new array**.

Mistake 4

Expecting deep copying.

```
const users = [
  {
    name: "John"
  }
];

const copy = users.concat();
```

Objects are still copied by **reference**.

Interview Questions

What does `concat()` do?

It combines two or more arrays into a new array.

Does `concat()` modify the original array?

No.

What does `concat()` return?

A new merged array.

Can `concat()` merge multiple arrays?

Yes.

```
arr1.concat(arr2,arr3);
```

Can `concat()` combine values and arrays?

Yes.

```
arr.concat(4,5);
```

Which method modifies the original array?

```
push()
```

or

```
splice()
```

Which method creates a new merged array?

```
concat()
```


Summary Table

Feature	Description
Method	<code>concat()</code>
Purpose	Merge arrays
Parameters	Arrays or values
Returns	New merged array
Changes Original Array	No
Supports Multiple Arrays	Yes
Common Uses	API Data, Shopping Cart, Student List, Products

Quick Comparison

Method	Purpose	Returns	Changes Original Array
<code>concat()</code>	Merge arrays	New Array	No
<code>push()</code>	Add to end	New Length	Yes
<code>splice()</code>	Add/Remove/Replace	Removed Elements	Yes

Final Notes

The `concat()` method is one of the safest ways to merge arrays because it **never modifies the original arrays**. It is widely used in:

- React state updates
- API response merging
- Shopping cart systems
- Student management systems
- Product catalogs
- Data processing

Mastering `concat()` is essential before learning advanced array methods like `flat()` , `map()` , `filter()` , and `reduce()` .